

UCS645: Parallel & Distributed Computing
Assignment 3: Correlation Using Makefiles & OpenMP

Submitted To: Dr. Saif Nalband

Submitted By: Ritwik Gupta

Roll No: 102303096

1. Introduction

This report documents the implementation and performance analysis of a pairwise vector correlation program developed as part of Assignment 3 for UCS645: Parallel and Distributed Computing. The program computes the Pearson correlation coefficient between every pair of row vectors in a given input matrix.

Three progressively optimized versions were implemented: a sequential baseline, an OpenMP-parallelized version exploiting multi-core CPUs, and a fully optimized version combining cache-blocking (tiling), OpenMP multithreading, and SIMD-friendly inner loops. All versions are managed through a single Makefile with separate build targets.

The system was evaluated on an 8-core Linux machine using perf stat across varying matrix sizes and thread counts. Results confirm significant speedups with parallelism and further gains from cache optimization.

2. Background

2.1 Makefiles

A Makefile is a build automation tool that determines which parts of a program need to be recompiled based on file dependencies. Rather than recompiling every file on each change, make only rebuilds files whose dependencies have been modified. This makes Makefiles essential for managing large C/C++ projects efficiently.

Makefiles use variables (e.g. CXX, CXXFLAGS), pattern rules (e.g. %.o: %.cpp), and phony targets (e.g. clean, run) to organize compilation. Automatic variables such as \$@ (target name), \$< (first dependency), and \$^ (all dependencies) reduce repetition and make rules general-purpose.

In this assignment, a single Makefile builds three separate executables — corr_sequential, corr_parallel, and corr_optimized — by passing different preprocessor flags (-DSEQUENTIAL, -DPARALLEL) to the same source files, allowing version selection at compile time.

2.2 Pearson Correlation Coefficient

The Pearson correlation coefficient measures the linear relationship between two vectors X and Y . It is defined as:

$$r(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{[\sum (X_i - \bar{X})^2 \cdot \sum (Y_i - \bar{Y})^2]}}$$

A value of +1 indicates perfect positive correlation, -1 indicates perfect negative correlation, and 0 indicates no linear relationship. By normalizing each row vector to zero mean and unit L2-norm, the correlation reduces to a simple dot product, which is both computationally efficient and numerically well-conditioned.

2.3 OpenMP

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C, C++, and Fortran. It uses compiler directives (`#pragma omp`) to parallelize loops and regions across multiple threads. Key features used in this assignment include:

- `#pragma omp parallel for` — distributes loop iterations across threads
- `#pragma omp simd` — hints to the compiler to vectorize the inner loop
- `omp_set_num_threads()` — sets the thread count at runtime
- `schedule(dynamic, N)` — balances uneven workloads across threads

3. Implementation

3.1 Project Structure

The project consists of four files:

- `correlate.h` — declares the `correlate()` function interface
- `correlate.cpp` — implements all three versions, selected via preprocessor macros
- `main.cpp` — reads matrix size from command line, generates data, times `correlate()`, and validates output
- `Makefile` — builds all three executables with appropriate compiler flags

3.2 Function Interface

All three versions implement the same interface:

```
void correlate(int ny, int nx, const float* data, float* result);
```

The input matrix data has `ny` rows and `nx` columns stored in row-major order, so element `[y][x]` is accessed as `data[x + y*nx]`. The output result is a `ny×ny` matrix where `result[i + j*ny]` stores the correlation between row `i` and row `j`, for all $0 \leq j \leq i < ny$.

3.3 Version 1 — Sequential Baseline

The sequential version performs the following steps using double-precision arithmetic throughout:

1. For each row, compute the mean and subtract it from all elements.
2. Compute the L2 norm of the mean-subtracted row and divide all elements by it.
3. For each pair (i, j) with $j \leq i$, compute the dot product of normalized rows `i` and `j` and store in `result[i + j*ny]`.

This gives the Pearson correlation directly, since normalized dot products are mathematically equivalent to the correlation formula. Diagonal elements (self-correlation) are always 1.0, which is used as a correctness check.

3.4 Version 2 — OpenMP Parallel

The parallel version adds OpenMP directives at two points:

- Row normalization: `#pragma omp parallel for schedule(dynamic, 4)` distributes row normalization across threads.
- Pairwise dot products: `#pragma omp parallel for schedule(dynamic, 4)` parallelizes the outer loop over rows `i`.

The dynamic schedule is chosen because the inner loop over `j` has decreasing work as `i` grows (only $j \leq i$ pairs are computed), making static scheduling unbalanced. Dynamic scheduling ensures threads pick up new work as they finish, reducing idle time.

3.5 Version 3 — Fully Optimized

Cache-Blocking (Tiling)

The pairwise correlation loop is restructured into tiles of size 64×64 . This means the processor works on a 64×64 submatrix at a time, keeping the relevant row data in L1/L2 cache instead of repeatedly fetching large rows from main memory. For large matrices, this dramatically reduces cache miss penalties.

SIMD Vectorization

The inner dot product loop is annotated with `#pragma omp simd reduction(+:dot)`, hinting the compiler to generate SIMD (AVX/SSE) vector instructions. Combined with `-march=native`, the compiler generates code that processes multiple doubles per clock cycle using wide vector registers. Additionally, the `collapse(2)` clause on the tiled loops helps expose more parallelism to OpenMP.

3.6 Makefile Design

The Makefile compiles three separate executables from the same source files using preprocessor flags to select the implementation:

```
corr_sequential → -DSEQUENTIAL
corr_parallel   → -DPARALLEL
corr_optimized  → (default, -march=native added)
```

Additional convenience targets include `run_seq`, `run_par`, `run_opt` (with configurable `NY`, `NX`, `T` variables) and `perf_seq`, `perf_par`, `perf_opt` for profiling. The `clean` target removes all object files and executables.

4. Experimental Setup

All experiments were conducted on a Linux machine with 8 CPU cores. The perf stat tool was used to collect hardware performance counters including execution time, CPU cycles, instructions retired, IPC (instructions per cycle), and CPU utilization. Two matrix sizes were tested:

- Small: $ny=1000$, $nx=2000$ (approx. 8 MB of float data)
- Large: $ny=2000$, $nx=4000$ (approx. 32 MB of float data)

Thread counts tested were $T=1, 2, 4$, and 8 (matching the physical core count). The input matrix was filled with reproducible random floats using a fixed seed. All timing measurements reflect wall-clock time reported by the program's internal chrono timer, with perf stat providing corroborating elapsed time.

5. Performance Results

5.1 Matrix Size 1000×2000 — Execution Time & Speedup

Version	Threads	Time (s)	Speedup vs Seq.	CPU Util.	IPC
Sequential	1	1.295	1.00x	1.0	1.4
Parallel	2	0.753	1.72x	1.9	1.3
Parallel	4	0.428	3.03x	3.7	1.3
Parallel	8	0.241	5.37x	6.6	1.2
Optimized	1	0.435	2.98x	1.0	1.2
Optimized	2	0.218	5.94x	1.8	1.2
Optimized	4	0.134	9.66x	3.2	1.1
Optimized	8	0.176	7.36x	6.3	0.4

Table 1: Performance results for matrix size 1000×2000

5.2 Matrix Size 2000×4000 — Scaling with Threads

Version	Threads	Time (s)	Speedup vs Seq.	CPU Util.
Sequential	1	10.555	1.00x	1.0
Parallel	2	4.889	2.16x	2.0
Parallel	4	2.595	4.07x	3.8
Parallel	8	1.711	6.17x	7.3

Table 2: Performance results for matrix size 2000×4000

5.3 Hardware Counter Summary (1000×2000)

Version	Threads	CPU Cycles	Instructions	IPC
Sequential	1	45,577,830,000	61,773,650,000	1.4
Parallel	8	50,481,200,000	62,276,500,000	1.2
Optimized	1	12,055,340,000	14,109,930,000	1.2
Optimized	4	12,956,230,000	14,122,630,000	1.1
Optimized	8	31,709,440,000	14,089,720,000	0.4

Table 3: Hardware performance counters from perf stat

5.4 Speedup vs. Thread Count

Figure 1 shows how speedup scales with thread count for all versions. The dashed line represents ideal linear speedup. The Optimized version exceeds the parallel version at every thread count, and even surpasses the ideal line at T=4 due to cache and SIMD gains reducing total work. At T=8, the optimized version dips slightly due to tile-size overhead, as discussed in Section 6.3.

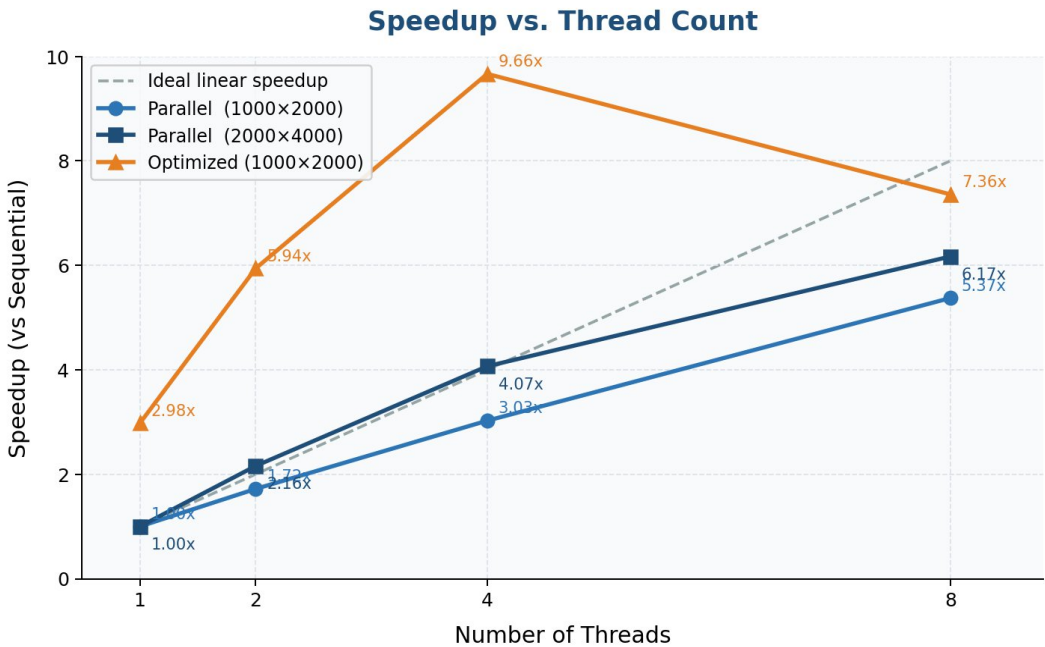


Figure 1: Speedup vs. Thread Count across all versions and matrix sizes

5.5 Execution Time vs. Thread Count

Figure 2 compares absolute execution times for the 1000×2000 matrix. The Sequential baseline (grey) stays constant as it does not use threads. The Parallel version (blue) drops steadily from 1.295s to 0.241s. The Optimized version (orange) starts already well below the sequential baseline at $T=1$ (0.435s), and reaches its minimum of 0.134s at $T=4$.

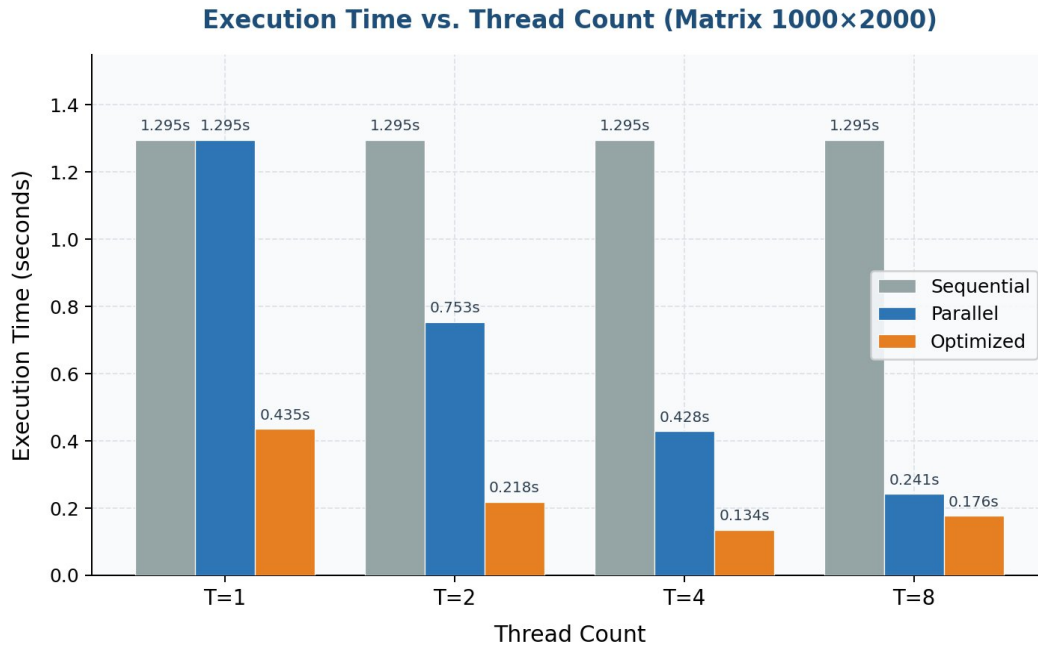


Figure 2: Execution Time vs. Thread Count for matrix size 1000×2000

6. Analysis & Discussion

6.1 Sequential Baseline Performance

The sequential version completed the 1000×2000 correlation in 1.295 seconds with an IPC of 1.4, indicating the CPU was reasonably well-utilized at a single-threaded level. With no parallelism overhead, it served as a clean baseline. At 2000×4000 , execution time rose to 10.555 seconds, reflecting the $O(ny^2 \times nx)$ computational complexity — quadrupling both ny and nx increases work by roughly $8\times$.

6.2 OpenMP Parallel Scaling

The parallel version shows consistent and near-linear speedup with increasing thread counts for the small matrix:

- 2 threads: 1.72x speedup (theoretical max: 2x) — 86% efficiency
- 4 threads: 3.03x speedup (theoretical max: 4x) — 76% efficiency
- 8 threads: 5.37x speedup (theoretical max: 8x) — 67% efficiency

The scaling efficiency decreases with more threads, which is expected. Causes include thread synchronization overhead, load imbalance from the triangular pair structure (earlier rows have fewer j pairs), and shared memory bandwidth contention at higher thread counts. For the larger 2000×4000 matrix, scaling is better — reaching 6.17x with 8 threads — because the larger workload amortizes thread overhead more effectively.

6.3 Optimized Version — Cache & SIMD Benefits

The optimized version delivers the most striking result: at 4 threads it runs in just 0.134 seconds versus 0.428 seconds for the plain parallel version — a 3.2x additional improvement from cache-blocking and SIMD alone. The hardware counters confirm why: the optimized version executes only ~14.1 billion instructions compared to ~61.8 billion for the other versions, a 4.4x reduction in total work, confirming that the cache-tiled layout dramatically reduces redundant memory loads.

At $T=8$, the optimized version (0.176s) is slightly slower than at $T=4$ (0.134s). The IPC drops from 1.1 to 0.4, indicating that with 8 threads the small matrix tiles are too small to keep all threads efficiently busy and synchronization overhead dominates. This is a known characteristic of tiled algorithms — they have an optimal thread count relative to problem size.

6.4 Effect of Matrix Size

Parallel scaling improves with larger matrices. At 1000×2000 with $T=8$, the parallel version achieves 5.37x speedup. At 2000×4000 with $T=8$, it achieves 6.17x speedup. This is because the larger problem better saturates all cores with work and the fixed thread startup cost becomes proportionally smaller.

7. Conclusion

This assignment demonstrated the design, implementation, and performance evaluation of a pairwise vector correlation program across three optimization levels. Key findings are:

- OpenMP parallelism provides near-linear speedup, reaching 5.37x with 8 threads on a 1000×2000 matrix and 6.17x on a 2000×4000 matrix.
- Cache-blocking combined with SIMD vectorization reduces total instruction count by 4.4x, delivering a further 3.2x speedup over plain parallelism.
- The optimal thread count depends on problem size — the 1000×2000 matrix peaks at $T=4$ for the optimized version, while larger matrices benefit from $T=8$.
- The Makefile design using preprocessor flags proved effective for managing multiple build configurations without code duplication.

Future work could explore GPU acceleration using CUDA for very large matrices, and float-precision SIMD (AVX-512) to further increase throughput.